



Chapter 3.2

Basic Principles of ANNs and Their Early Structures



Basic Principles of ANN Design

- First formulated by McCulloch and Pitts in 1943
- Assumptions are
 1. The activity of a neuron (ANN) is all-or-nothing.(the neuron is a binary element)
 2. A certain fixed number of synapses larger than 1 must be excited within a given interval of neural addition for a neuron to be excited.
 3. The only significant delay within the neural system is the synaptic delay.
 4. The activity of any inhibitory synapse absolutely prevents the excitation of the neuron at that time.
 5. The structure of the interconnection network does not change over time.
- They do not all apply to today's state-of-the-art of ANN design.



Hebbian Learning Law

- When an axon of cell A is **near-enough to excite cell B** and when it **repeatedly and persistently** takes part in ring it, then some growth process or metabolic change takes place in one or both these cells such that the **efficiency of cell A** [Hebb, 1949] is increased" (i.e. the **weight of the contribution** of the output of cell A to the above ring of cell B is **increased**).
- Example:
 - Cell S causes salivation
 - Cell F is excited by the sight of food
 - So Cell S excited by Cell F
 - Cell L excited by hearing a bell ring
 - Cell L connects to cell S
 - After repeated ring of S by cell F while also cell L is ring, then L will eventually be able to cause S to re without having cell F fire.
 - This will be due to the eventual increase in the weight of the input from cell L into cell S
 - Cells L and S play the role of cells A, B respectively of Hebbian Learning Law
- **The Hebbian rule need not be employed in all ANN designs**



Basic Network Structures

1. Historically, the earliest ANNs are The **Perceptron**, proposed by the psychologist **Frank Rosenblatt** (Psychological Review, 1958).
2. The **Artron** (Statistical Switch-based ANN) due to **R. Lee** (1950s).
3. The **Adaline** (Adaptive Linear Neuron, due to **B. Widrow**, 1960). This artificial neuron is also known as the ALC (adaptive linear combiner), the ALC being its principal component. **It is a single neuron, not a network.**
4. The **Madaline** (Many Adaline), also due to **Widrow** (1988). This is an ANN (network) formulation based on the Adaline above. **Perceptron**, are **common building blocks** in most later ANN developments.

Basic Network Structures (Cont.)



5. The **Back-Propagation** network-A multi-layer Perceptron-based ANN, giving an elegant solution to hidden-layers learning [Rumelhart et al., 1986 and others].
5. The **Hopeld** Network, due to **John Hopeld** (1982).
 - These two are different from the earlier four ANNs in many important aspects, especially in its **recurrent feature of feedback** between neurons.
7. The Counter-Propagation Network [Hecht-Nielsen, 1987] - where **Kohonen's Self-Organizing Mapping** (SOM) is utilized to facilitate unsupervised learning ("absence of a teacher").

The Perceptron's Input-Output Principles

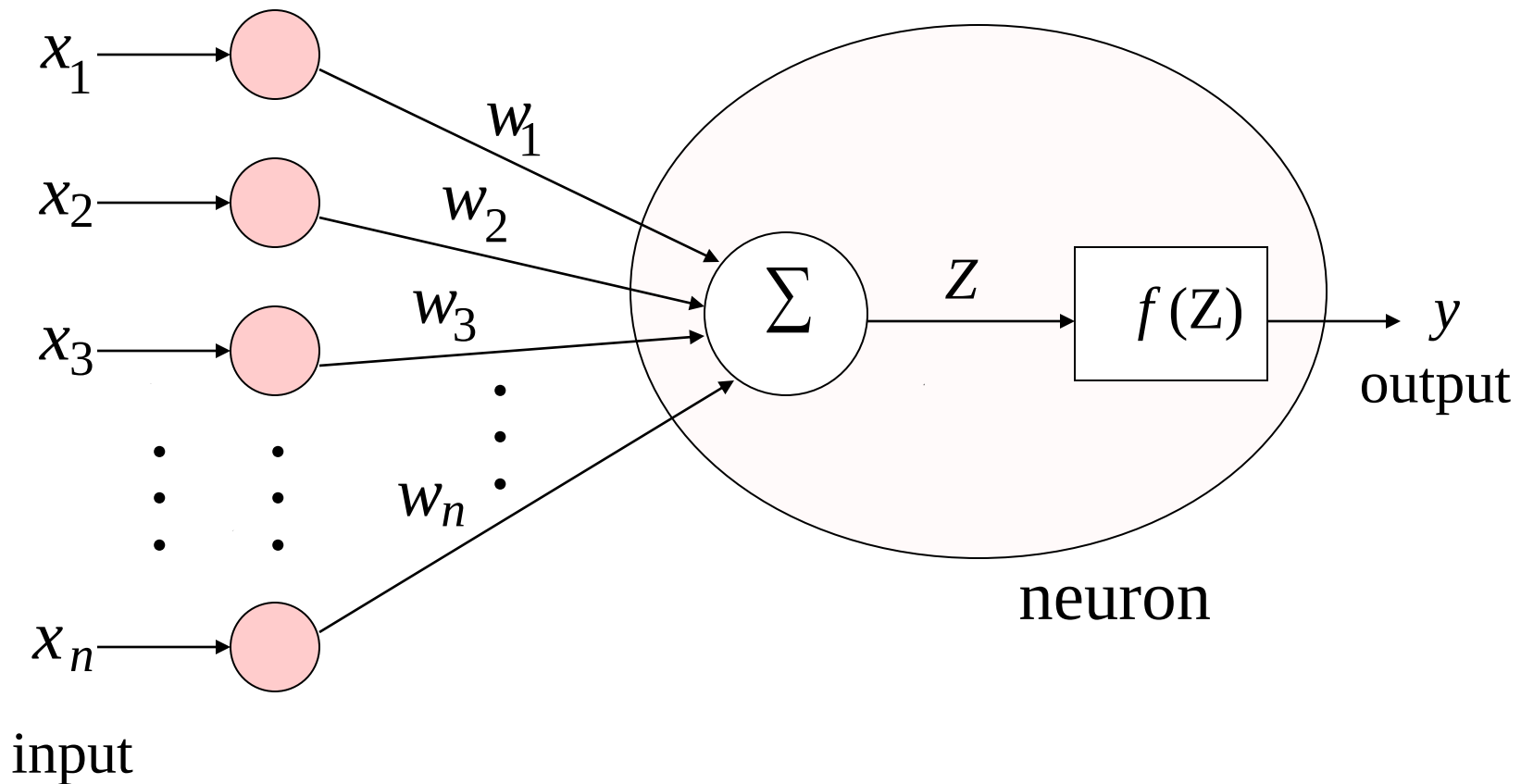


- Proposed by [Rosenblatt, 1958](#).
- Basic building block of nearly all ANNs.
- $Z = \sum_{i=1}^N w_i x_i$ and $y = f_N(Z)$

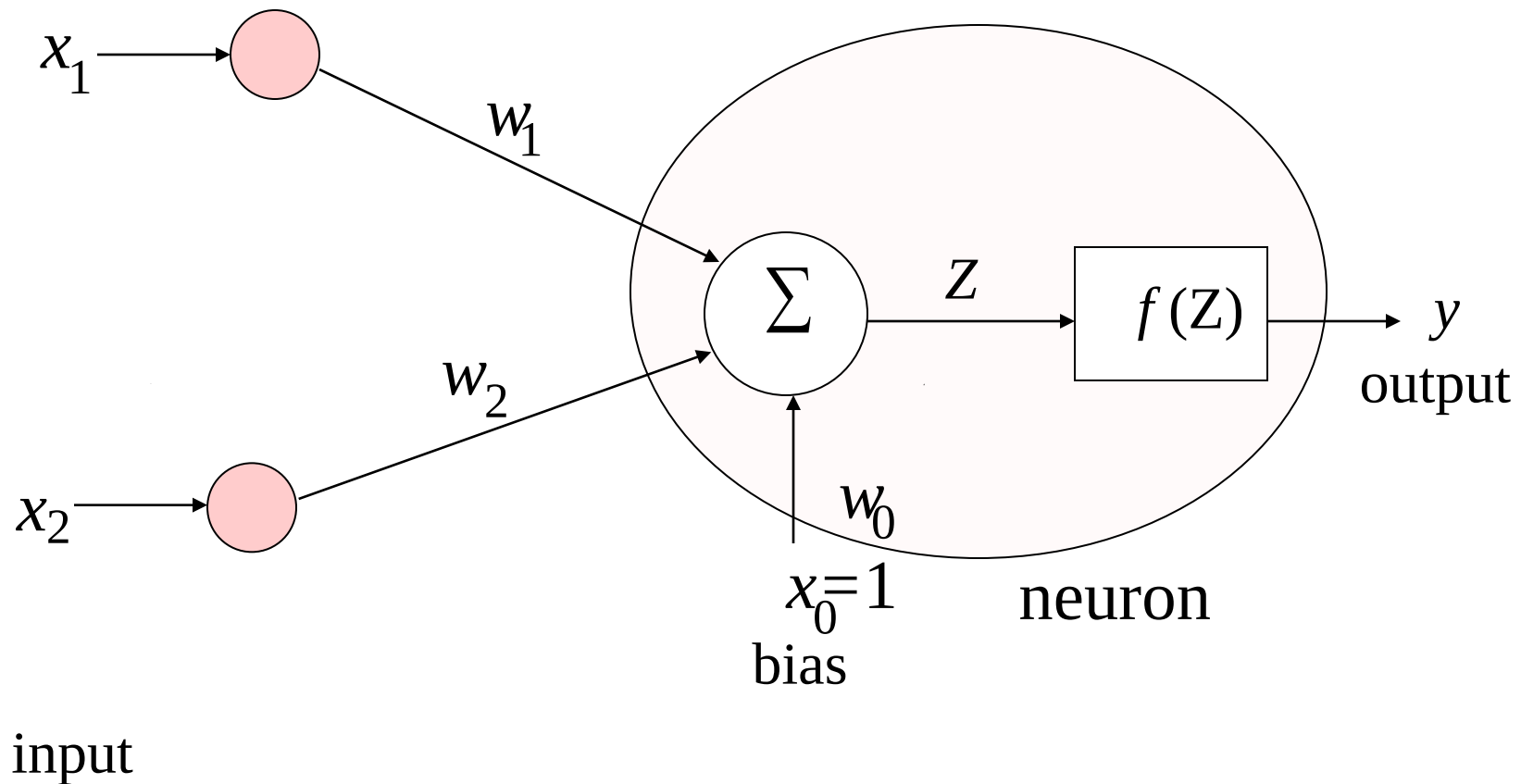
where w_i is the weight at the inputs x_i where Z is the node (summation) output and f_N is a nonlinear operator.

- $y = \text{sign}(z)$ i.e. $y=f(z)=1$ if $z>0$ and $f(z)=-1$ if $z\leq 0$
- The output of the network thus is either +1 or -1, depending on the input.

Perceptron's Input-Output Principles (Cont.)



Perceptron's Input-Output Principles (Cont.)



Perceptron's Input-Output Principles (Cont.)



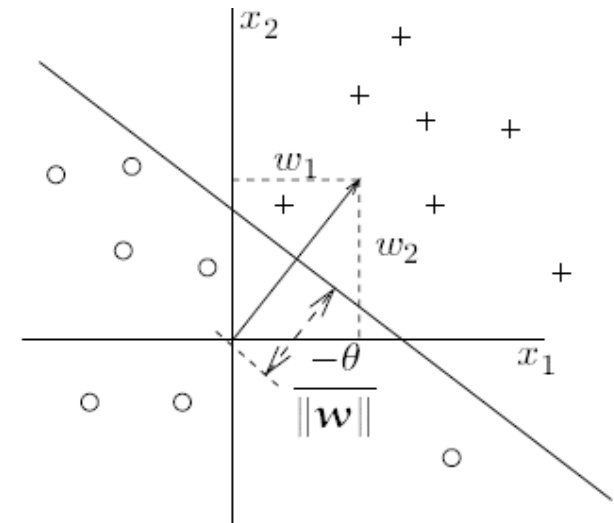
- The network can now be used for a classification task: it can decide whether an input pattern belongs to one of two classes.
- If the total input is positive, the pattern will be assigned to class +1, if the total input is negative, the sample will be assigned to class -1.
- The separation between the two classes in this case is a straight line, given by the equation:

$$w_1x_1 + w_2x_2 + w_0 = 0$$

- The single layer network represents a **linear discriminant function**.

$$x_2 = -w_1/w_2 x_1 - w_0/w_2$$

weights determine the slope of the line
and the bias determines the 'offset'
considering $w_0 = \theta$



Perceptron's Input-Output Principles (Cont.)



- How do we learn the weights and biases in the network?
- Iterative procedures are there.
- Weight the new value is computed by adding a correction to the old value

$$w_i(t + 1) = w_i(t) + \Delta w_i(t)$$

$$w_o(t + 1) = w_o(t) + \Delta w_o(t)$$

- How do we compute $\Delta w_i(t)$ and $\Delta w_o(t)$ in order to classify the learning patterns correctly?

Perceptron learning rule and convergence theorem



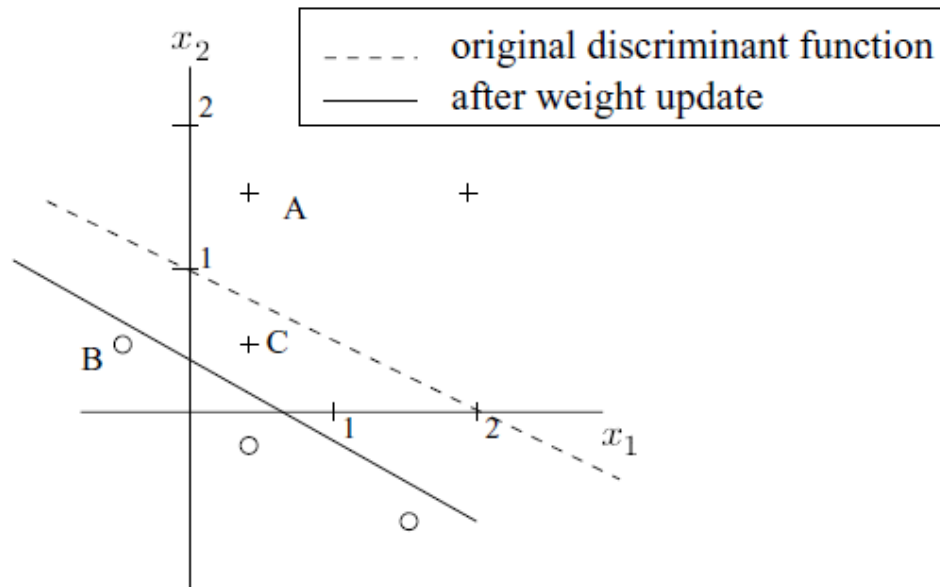
- We have a set of learning samples consisting of an input vector x and a desired output $d(x)$
- Classification task the $d(x)$ is usually $+1$ or -1 .
- Perceptron learning rule is very simple and can be stated as follows:
 1. Start with random weights for the connections.
 2. Select an input vector x from the set of training samples.
 3. If $y \neq d(x)$ (the perceptron gives an incorrect response), modify all connections w_i according to: $\Delta w_i = d(x)x_i$;
 4. Go back to 2.



Example of the Perceptron learning rule

- Initially: $w_1 = 1; w_2 = 2; w_0 = -2$.
- For sample A, $x = (0.5; 1.5)$ and target value $d(x) = +1$
- The network output is +1, so no weights are adjusted.
- For sample B, $x = (-0.5; 0.5)$ and target value $d(x) = -1$
- The network output is -1, so no weights are adjusted.
- For sample C, $x = (0.5; 0.5)$ and target value $d(x) = +1$
- The network output is -1, so weights are to be adjusted.
- According to the perceptron learning rule, the weight changes are: $\Delta w_1 = 0.5, \Delta w_2 = 0.5, \Delta w_0 = 1$.
- The new weights are now: $w_1 = 1.5, w_2 = 2.5, w_0 = -1$, and sample C is classified correctly.

Example of the Perceptron learning rule (Cont.)

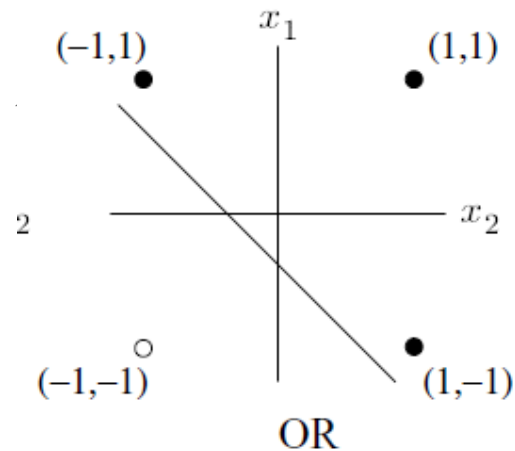
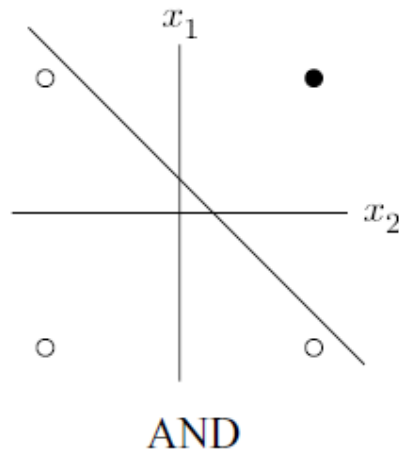


Discriminant function before and after weight update.



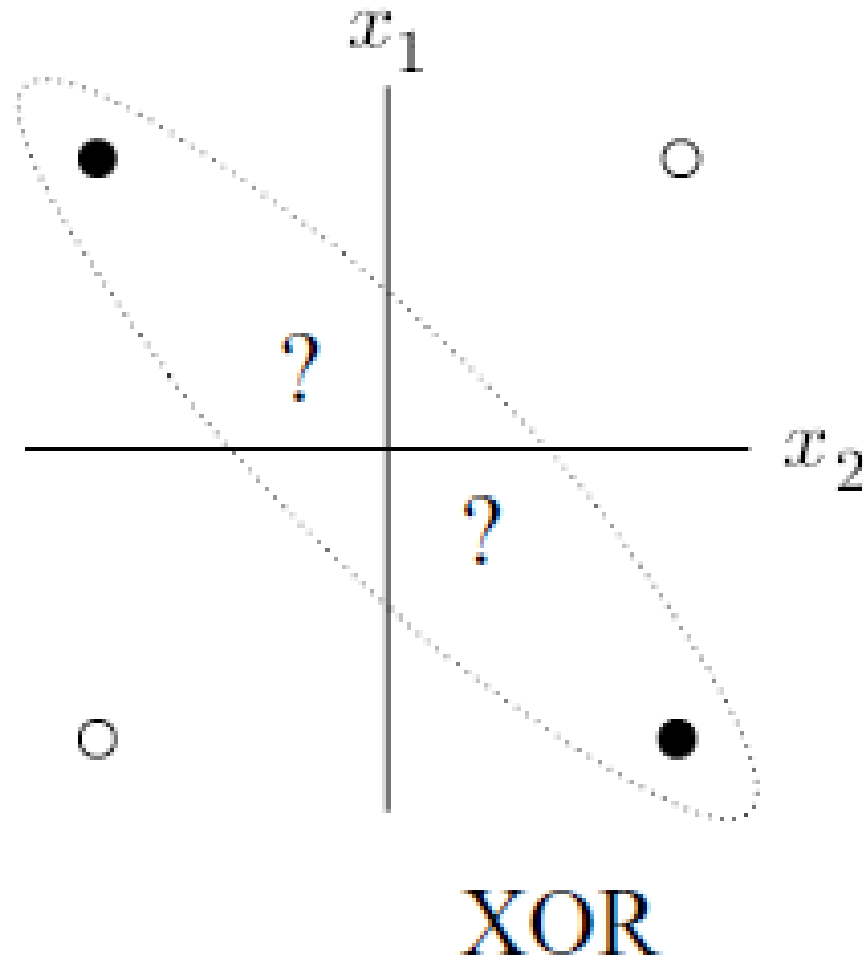
Exclusive-OR problem

- We have discussed two learning algorithms for single layer networks
- Limitations
- In a simple network with two inputs and one output, the net input is equal to: $s = w_1x_1 + w_2x_2 + \theta$.
- The output of the perceptron is zero when s is negative and equal to one when s is positive.





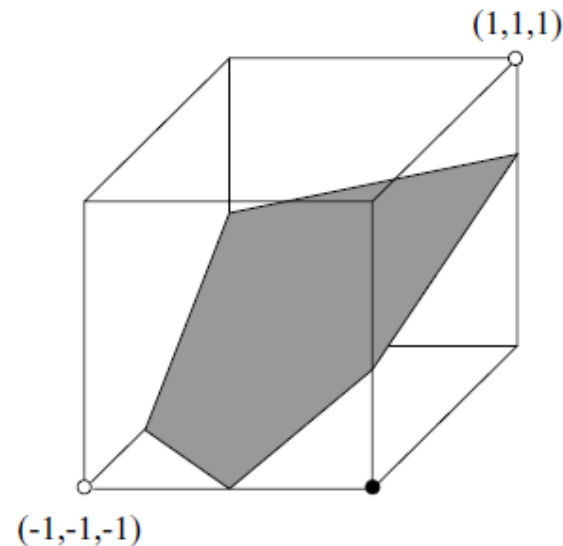
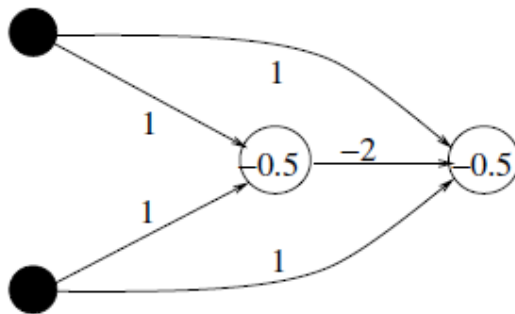
Exclusive-OR problem (Cont.)





Exclusive-OR problem (Cont.)

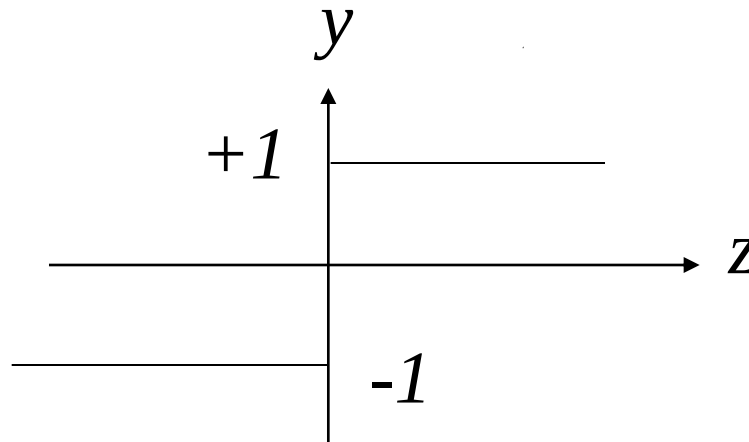
- **Minsky and Papert** first shows that a single layer perceptron cannot represent a simple exclusive-or function
- How can this problem be overcome?
- Minsky and Papert prove in their book that for binary inputs, any transformation can be carried out by **adding a layer** of predicates which are connected to all inputs.
- Introducing **hidden units**, thereby extending the network to a **multi-layer perceptron**.





The Adaline (ALC)

- The **Adaline (ADaptive LInear NEuron)** is proposed by **B. Widrow (1960)**
- Has the basic structure of a **bipolar Perceptron**.
- Involves **least error-square (LS)** weight training.
- $z = w_0 + \sum_{i=1}^n w_i x_i$, for $w_0 = 0$ $Z = \sum_{i=1}^n w_i x_i$
- Operator: a simple threshold element.
- $y = \text{sign}(Z)$ i.e $y=f(Z)=1$ if $Z>0$ and $f(Z)=-1$ if $z \leq 0$





The Adaline (ALC) (Cont.)

- An important generalization of the perceptron training algorithm was presented by Widrow and Hoff as the 'least mean square' (LMS) learning procedure, also known as the **delta rule**.
- The **main functional difference** with the perceptron training rule is the way the output of the system is used in the learning rule.
- The perceptron learning rule uses the output of the threshold function (either -1 or +1) for learning. The delta-rule uses the net output **without further mapping** into output values -1 or +1.
- Let input conductances are denoted by w_i , $i = 0, 1, \dots, n$, the input and output signals are x_i and y , respectively.
- Then the output of the central block is defined to be
$$Z = \sum_{i=1}^n w_i x_i + \theta \text{ where } \theta = w_0$$
- The purpose of this device is to yield a given value $y = d^p$ at its output when the set of values x_i^p $i = 0, 1, \dots, n$, is applied at the inputs.
- The problem is to determine the coefficients w_i , $i = 0, 1, \dots, n$.



Networks with linear activation functions: the delta rule

- For a single layer network with an output unit with a linear activation function the output is simply given by $Z = \sum_{i=1}^n w_i x_i + \theta$
- A set of training samples consisting of input values x^p and desired (or target) output values d^p
- If z^p is the actual output then output of the network differs from the target value d^p by $(d^p - z^p)$
- The delta-rule now uses a cost- or error-function based on these differences to adjust the weights.
- The error function, as indicated by the name least mean square, is the summed squared error.
- Total error $E = \sum_p E^p = 1/2 \sum_p (d^p - z^p)^2$

where the index p ranges over the set of input patterns and E^p represents the error on pattern p

Networks with linear activation functions: the delta rule(Cont.)



- The LMS procedure finds the values of all the weights that minimize the error function by a method called **gradient descent**.
- Change in the weight proportional to the negative of the derivative of the error as measured on the current pattern with respect to each weight.

$$\Delta_p w_i = -\gamma \partial E^p / \partial w_i \quad \partial E^p / \partial w_i = \partial E^p / \partial Z^p * \partial Z^p / \partial w_i$$

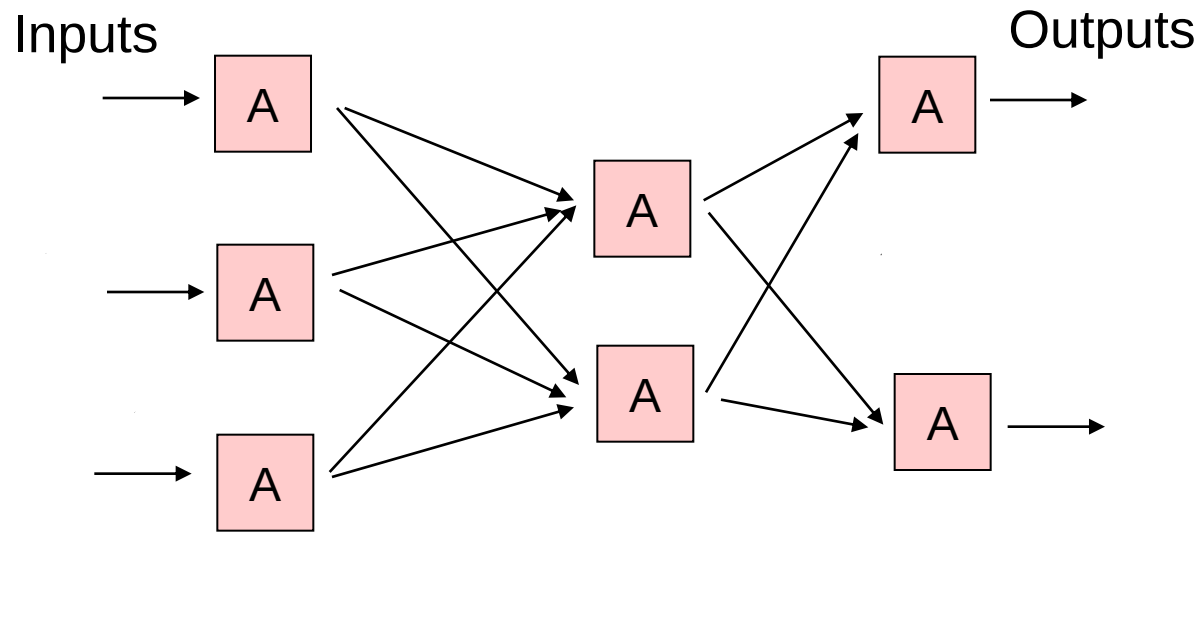
where γ is the learning rate.

- Because of the linear units $\partial Z^p / \partial w_i = x_i$ and $\partial E^p / \partial Z^p = -(d^p - z^p)$
- $\Delta_p w_i = \gamma (d^p - z^p) x_i$



Madaline

- The Madaline (Many Adaline) is a multilayer extension of the single-neuron bipolar Adaline to a network
- It is also due to [B. Widrow \(1988\)](#)
- The use of multiple Adaline helps counter the problem of non-linear separability.

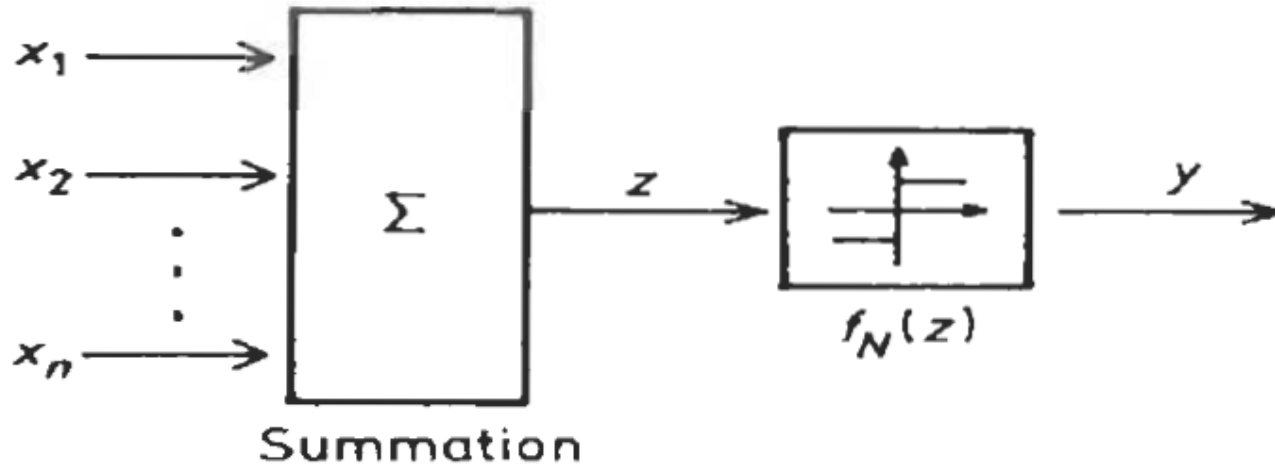


:Adeline Network



Madaline (Cont.)

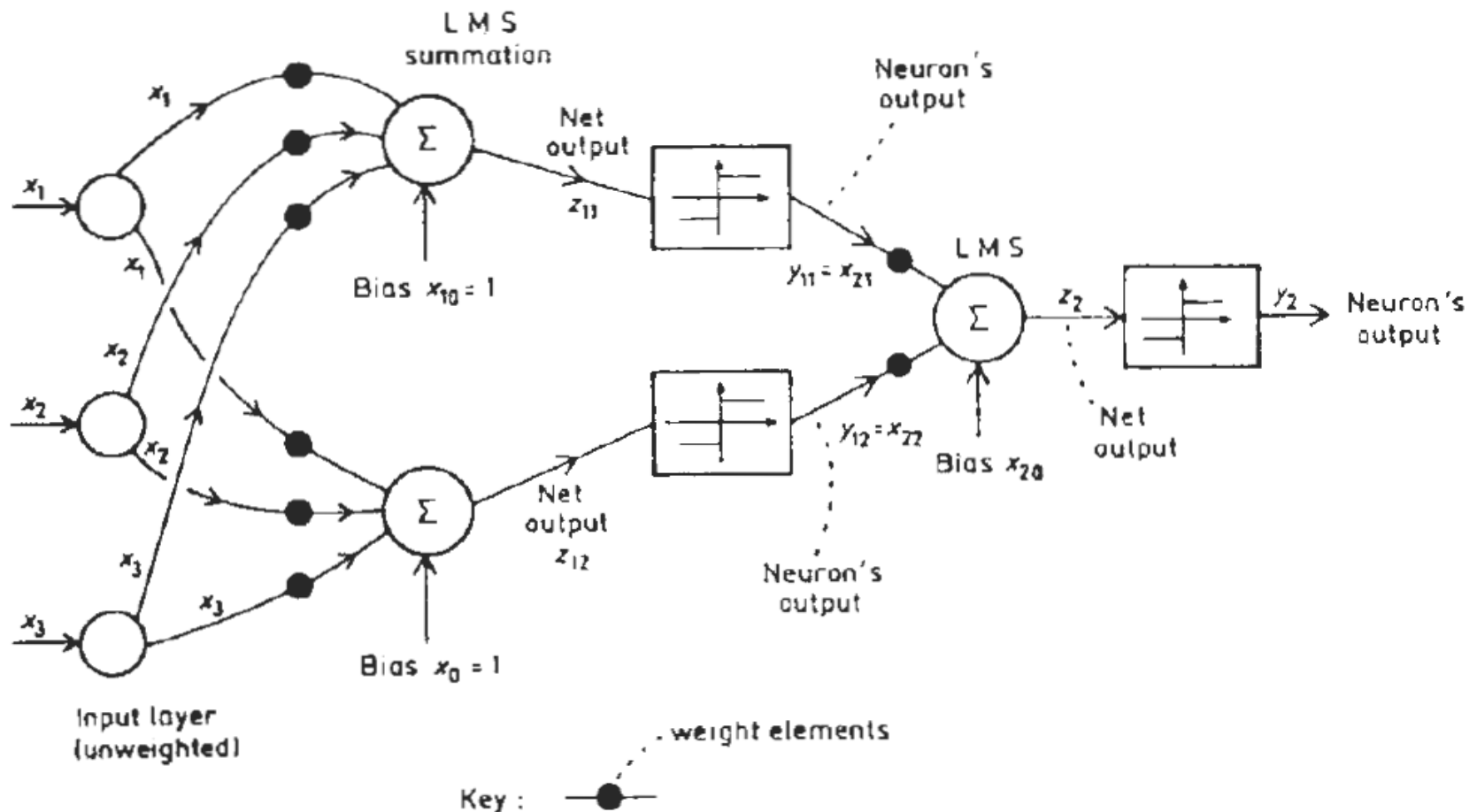
- A simple Adaline structure





Madaline (Cont.)

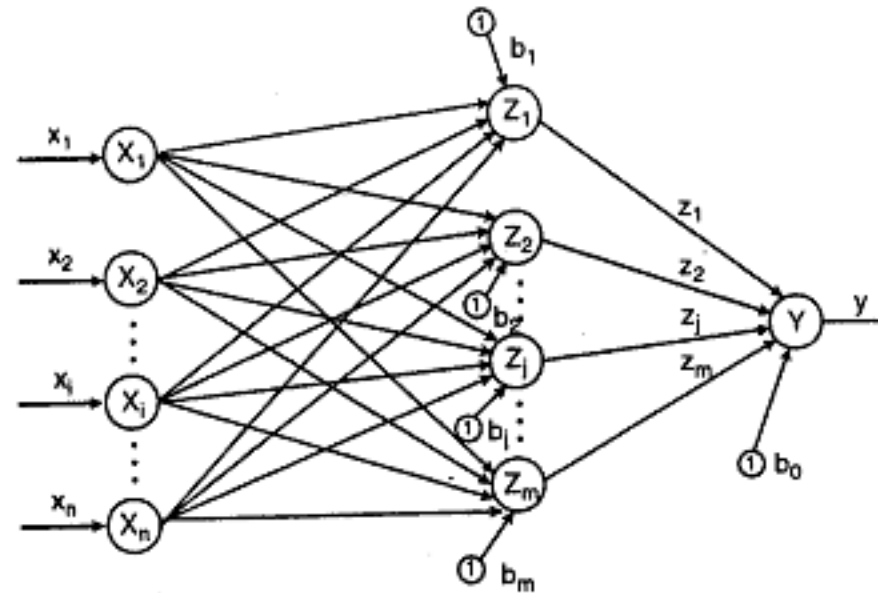
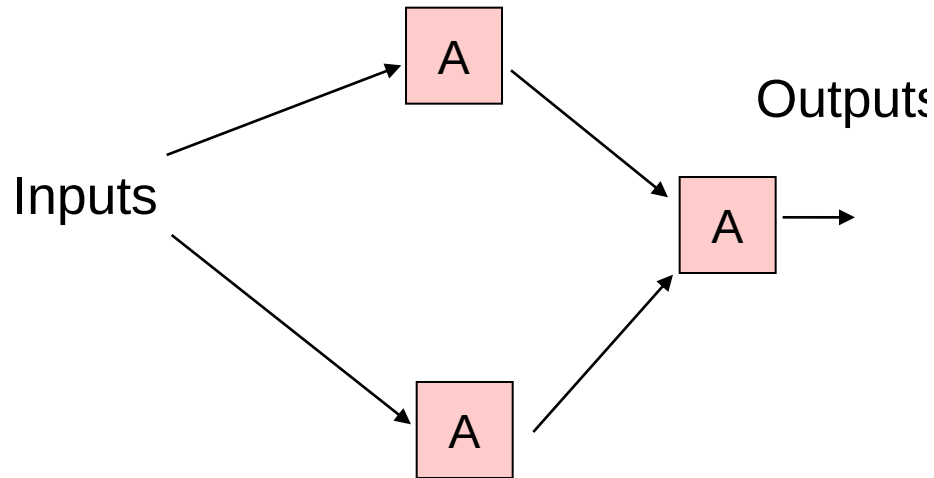
- The Madaline network of 2 layers.





Madaline (Cont.)

- The Madaline network of 2 layers.



Input layer

Adaline layer

Madaline layer



Madaline (Cont.)

- Figure shows 'n' units in the input layer, 'm' units in the ADALINE (1st layer) layer and one unit of MADALINE (2nd layer) layer. Each unit in the ADALINE, and MADALINE layer has a bias of excitation 1.
- All the weights that are connected from the ADALINE layer to the MADALINE are fixed, positive and have equal values.
- If more than half of the ADALINE output is '+1', then the output of the MADALINE is +1 else -1.
- The weights connected from the input layer to the ADALINE layer are adjusted during training process.
- During the updating procedure, the MADALINE output is first compared to target output. Since each is +1 or -1, there is either a match or not a match. If there is a match, nothing is changed. If there is no match, then some of the weights are updated.
- Specifically, the ADALINE unit whose sum was closest to zero in the wrong direction is adopted, if the corresponding target value is 1. If there is a mismatch and the target value is '-1', then all the weights connected to the ADALINE units that have positive net input are adopted.



Madaline Training

- For simplicity only one unit is assumed in the MADALINE layer.
- The inside layers are thus termed **hidden layers**.
- The activation functions for the ADALINE and MADALINE units are
$$f(Z)=+1 \text{ if } Z \geq 0 \text{ and } f(Z)=-1 \text{ if } Z < 0$$
- Training procedure known as **Madaline Rule II**, based on **Minimum Disturbance Principle**.
 1. All weights are initialized at low random values. Set the learning rate, γ small positive value greater than 0.
 2. While stopping condition is false, do steps 3 to 9.
 3. For each bipolar training pair $X:T$, do steps 4 to 8.
 4. Set the activations for the input units ($i = 1, 2, 3, \dots, n$).



Madaline Training (Cont.)

5. Compute the net input to each ADALINE unit ($j = 1, 2, 3, \dots, m$).

$$Z_j = b_j + \sum_{i=1}^n x_i w_{ij}$$

6. Compute the output of each ADALINE unit ($j = 1, 2, 3, \dots, m$).

$$Y_j = f(Z_j)$$

7. Determine the net input to the MADALINE unit and determine the output

$$Z = b_o + \sum_{j=1}^m Y_j$$

$$Y = f(Z)$$

8. Determine the error and update weights.

If $T = Y$, no weights are updated.

If $T \neq Y$, then



Madaline Training (Cont.)

If $T=1$, then update weights on the ADALINE unit, Z_j whose net input is closest to zero.

$$b_j(\text{new}) = b_j(\text{old}) + \gamma(1 - Z_j)$$

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \gamma(1 - Z_j)x_i \quad (i = 1, 2, 3, \dots, n).$$

If $T=-1$, then update weights on all units Z_j , that have positive net input.

$$b_j(\text{new}) = b_j(\text{old}) + \gamma(-1 - Z_j)$$

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \gamma(-1 - Z_j)x_i \quad (i = 1, 2, 3, \dots, n).$$

9. Test for stopping condition.

If there are no weight changes (or weight change has reached an acceptable level) or a specified maximum number of weight update iterations have been performed then stop, or else continue.



Hebb learning rule

- Hebb's (1949) original statement was:

“When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes place in firing it, some growth process or metabolic change takes place in one or both cells “such that A's efficiency, as one of the cells firing B, is increased.”

- Hebb's hypothesis may be stated as:

“Increase the value of weight if the output is active when the input associated with that weight is also active.”

- Stent [1973] rephrased the Hebb's hypothesis as:

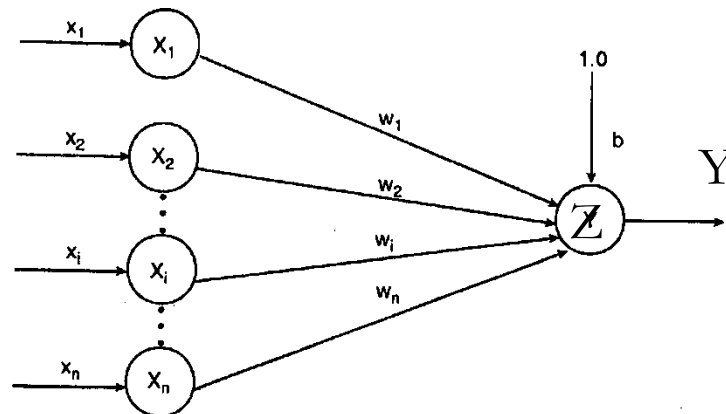
1. If two neurons on either side of a synapse (connection) are activated simultaneously, then the strength of that, synapse is selectively increased.
2. If two neurons on either side of a synapse are activated asynchronously, then synapse is selectively weakened or eliminated.

- For Hebbian rule, the weight vector increases proportional to the product of input and the learning signal, where the learning signal is equal to the neuron's output.



Hebb Net

- Based on Hebbian Learning rule, [McClelland and Rumelhart \(1988\)](#) developed the Hebb net.
- It is a single layer feedforward neural network
- This net has a fixed bias value of 1. The input and the output patterns must be of bipolar form.





Algorithm to Train Hebb Net

- Initialize all weights to zero, $w_i = 0$ (for $i = 1, 2, 3, \dots, n$) where n is the number of input neurons. Initialize the bias value to zero, $b = 0$.
- For each input vector, target output pair $X:T$, do steps 3 to 6.
- Set activations for input units with the input vector. x_i (for $i = 1, 2, 3, \dots, n$)
- Set the corresponding output value to the output neuron. $Y=T$
- Apply Hebb rule and modify weights.
$$w_i (\text{new}) = w_i (\text{old}) + x_i Y \quad (\text{for } i = 1, 2, 3, \dots, n)$$
- Adjust the bias.
$$b (\text{new}) = b (\text{old}) + Y$$



Thank you

Questions?